THE UNIVERSITY OF ALABAMA®

WHERE LEGENDS ARE MADE®

Binary Search Trees (BST)

To Implement Set and Dictionary ADTs
by Performing Binary Search

CS 201: Data Structures and Algorithms
Jiacheng Li, Ph.D.

Recall These ADTs

- Set
  - `void insert(ElementType x)`
  - `boolean find(ElementType x)`
  - `void remove(ElementType x)`
- Dictionary
  - `void insert(ElementType key, AnotherType val)`
  - `AnotherType find(ElementType key)`
  - `void remove(ElementType key)`

  - A **dictionary** acts as a function or mapping:
    - find(key) = val

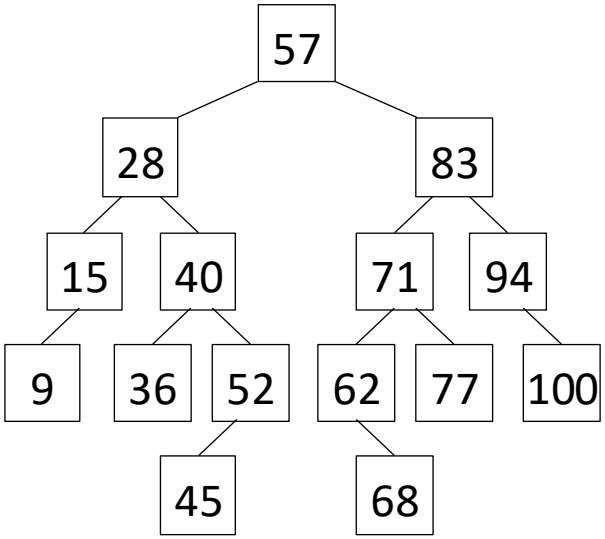
Implementations

- Linked lists (inefficient)
  - Insert in $\theta(n)$
  - Find in $\theta(n)$
  - Remove $\theta(n)$
- Hash tables (before)
- **Search trees**

Binary Search Trees

- If node X is in left subtree of node Y, then X.data ≤ Y.data
- If node Z is in right subtree of node Y, then Z.data ≥ Y.data

- ***Note:*** inorder traversal yields ascending order



BST Code

```
class Node {
  ElementType data;
  Node left, right;
  Node(ElementType x) {
    data = x;
    left = null;
    right = null;
  }
}
class BinarySearchTree {
  Node root;
  BinarySearchTree() {
    root = null;
  }
```

BST Code – find()

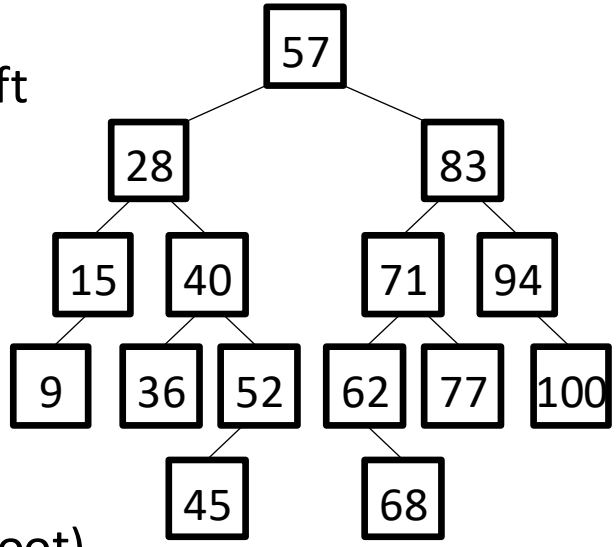
```
boolean find(ElementType x) {
  return find(x, root);      // Wrapper function
}

boolean find(ElementType x, Node p) {
  if (p == null) return false; // base case
  if (x == p.data) return true; // base case
  if (x < p.data) return find(x, p.left);
/*if (x > p.data)*/return find(x, p.right);
}
```

- *Remember:*
  - All nodes to the **left** are **less** than the current.
  - All nodes to the **right** are **greater** than the current.

find() – A High Level View

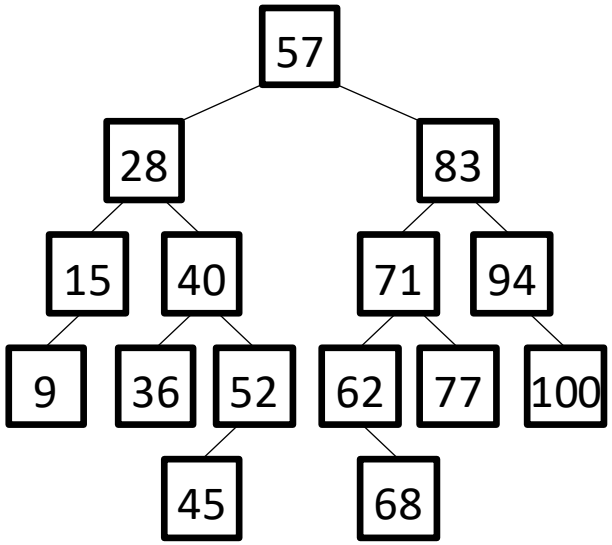
```
boolean find(ElementType x, Node p) {
  if (p == null) return false;
  if (x == p.data) return true;
  if (x < p.data) return find(x, p.left);
/*if (x > p.data)*/return find(x, p.right);
}
```

- Base cases
  1. The current "root" is the item – stop
  2. The current "root" is null – no items left
- Recursive cases
  - Divide the problem into subproblems of the same type
  1. Look left if the item is smaller
  2. Look right if the item is bigger
  - Solve the subtree (current "root") the exact same as the main tree (real root)



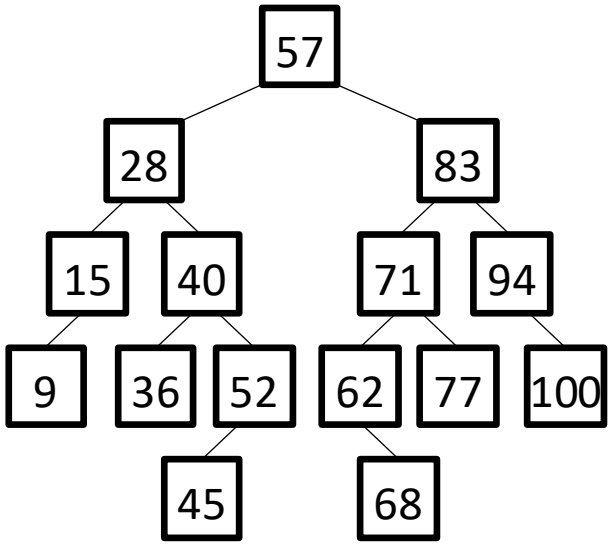
find(45) → true

```
boolean find(ElementType x, Node p) {
  if (p == null) return false;
  if (x == p.data) return true;
  if (x < p.data) return find(x, p.left);
/*if (x > p.data)*/return find(x, p.right);
}
```



find(93) → false

```
boolean find(ElementType x, Node p) {
  if (p == null) return false;
  if (x == p.data) return true;
  if (x < p.data) return find(x, p.left);
/*if (x > p.data)*/return find(x, p.right);
}
```

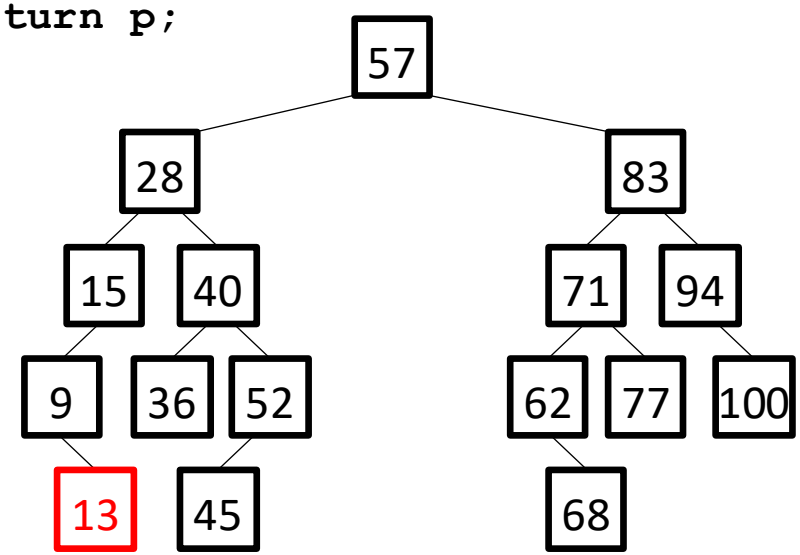


BST Code – insert()

```
void insert(ElementType x) {
  root = insert(x, root);
}

Node insert(ElementType x, Node p) {
  if (p == null) // end of tree
    return new Node(x);     // place new node
  if (x < p.data) // go left
    p.left = insert(x, p.left);
  else if (x > p.data) // go right
    p.right = insert(x, p.right);
  else // if (x == p.data)
    throw exception("Duplicate key");
  return p;
}
```

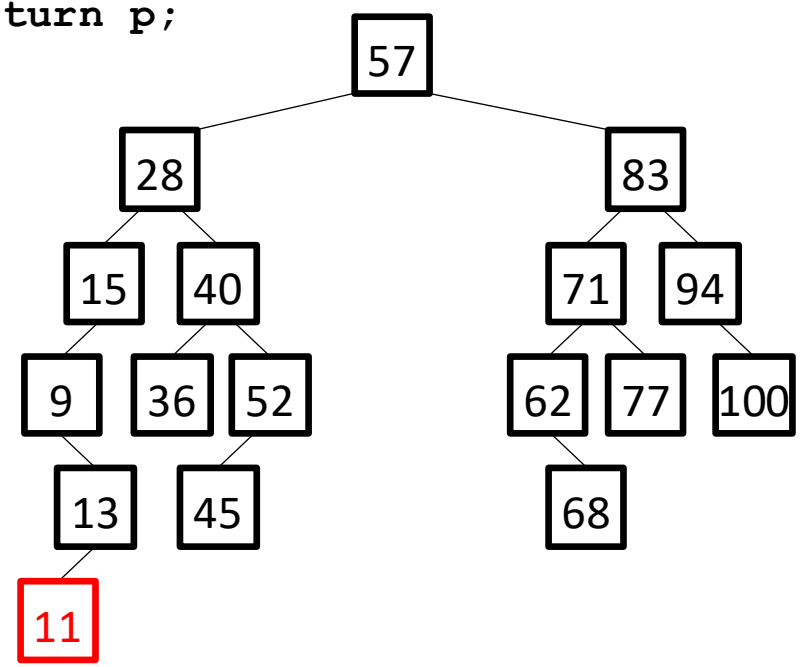
insert(13)

```
Node insert(ElementType x, Node p) {
  if (p == null) return new Node(x);   // end of tree
  if (x < p.data) p.left = insert(x, p.left);
  else if (x > p.data) p.right = insert(x, p.right);
  else throw exception("Duplicate key");
  return p;
}
```



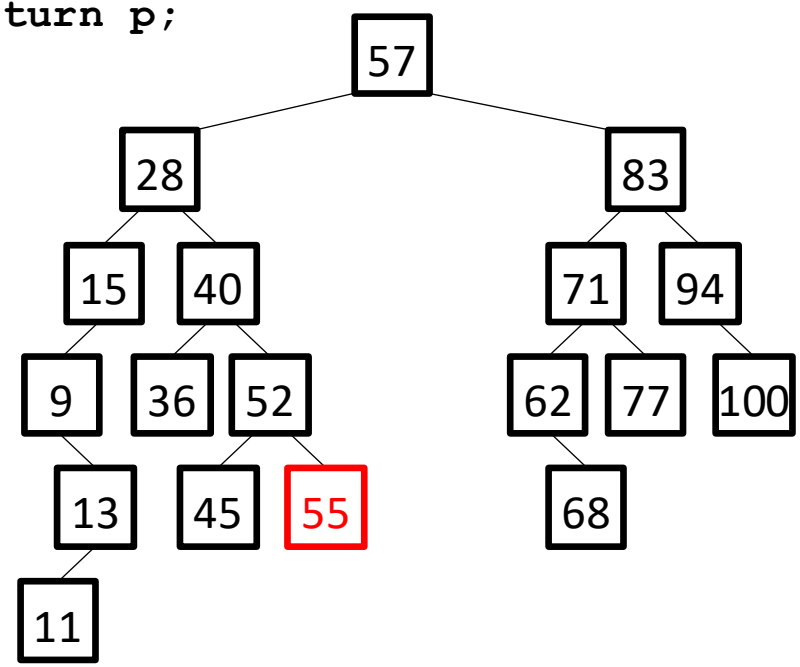
insert(11)

```
Node insert(ElementType x, Node p) {
  if (p == null) return new Node(x);
  if (x < p.data) p.left = insert(x, p.left);
  else if (x > p.data) p.right = insert(x, p.right);
  else throw exception("Duplicate key");
  return p;
}
```



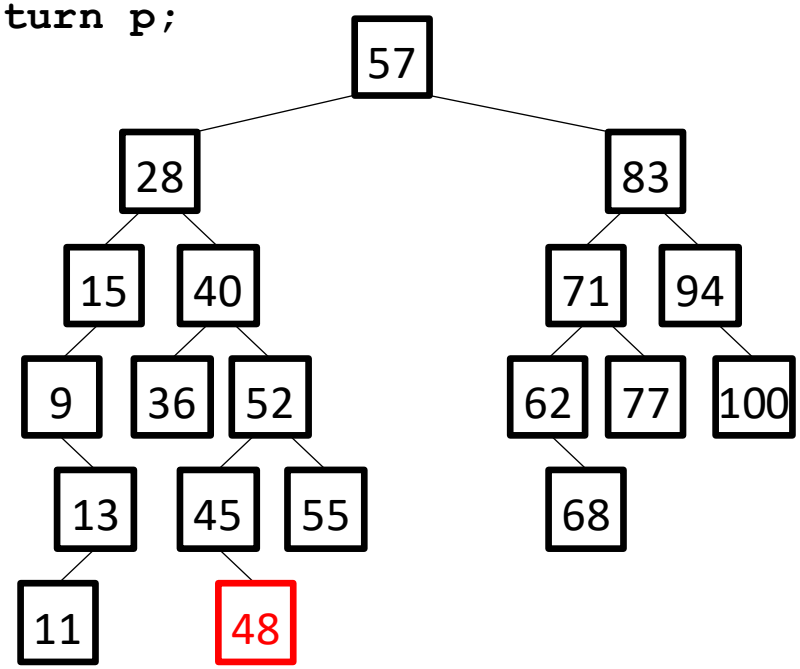
insert(55)

```
Node insert(ElementType x, Node p) {
  if (p == null) return new Node(x);
  if (x < p.data) p.left = insert(x, p.left);
  else if (x > p.data) p.right = insert(x, p.right);
  else throw exception("Duplicate key");
  return p;
}
```



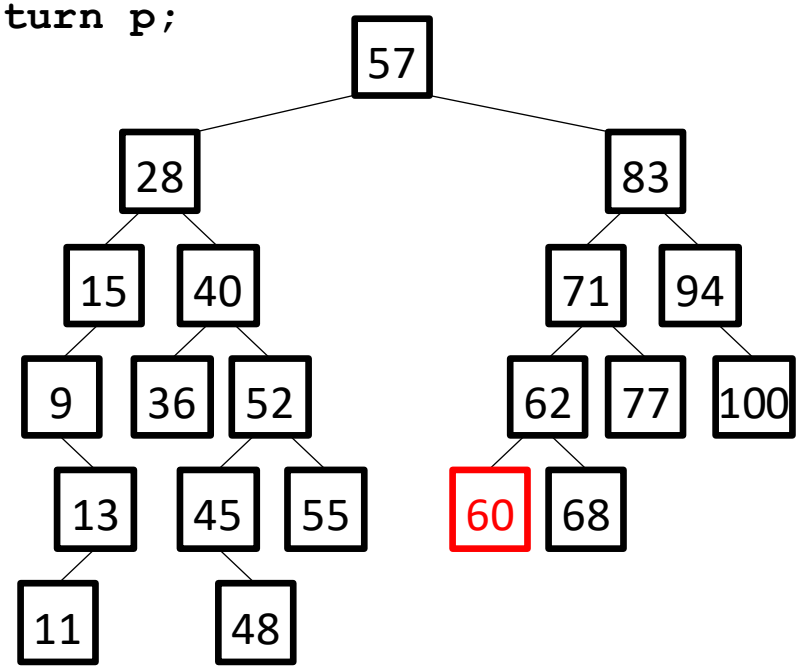
insert(48)

```
Node insert(ElementType x, Node p) {
  if (p == null) return new Node(x);
  if (x < p.data) p.left = insert(x, p.left);
  else if (x > p.data) p.right = insert(x, p.right);
  else throw exception("Duplicate key");
  return p;
}
```



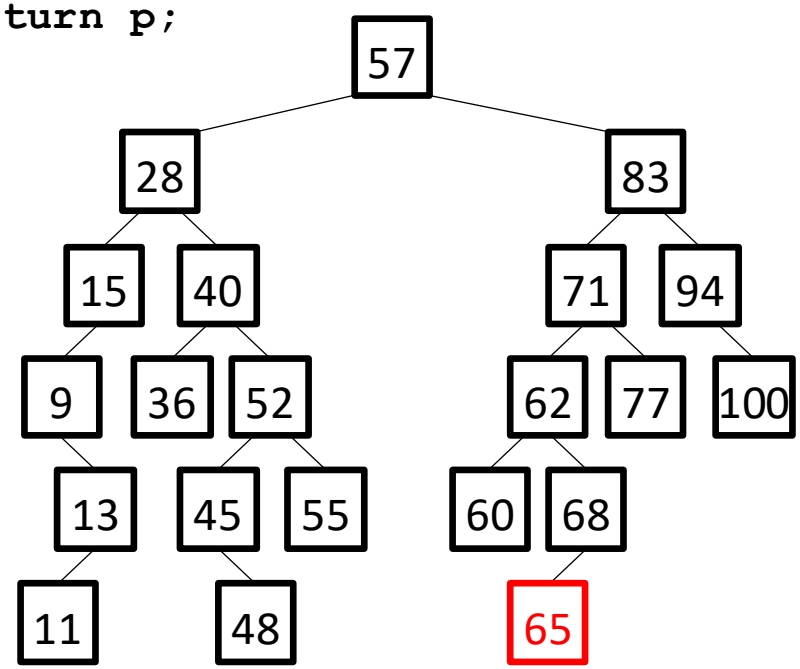
insert(60)

```
Node insert(ElementType x, Node p) {
  if (p == null) return new Node(x);
  if (x < p.data) p.left = insert(x, p.left);
  else if (x > p.data) p.right = insert(x, p.right);
  else throw exception("Duplicate key");
  return p;
}
```



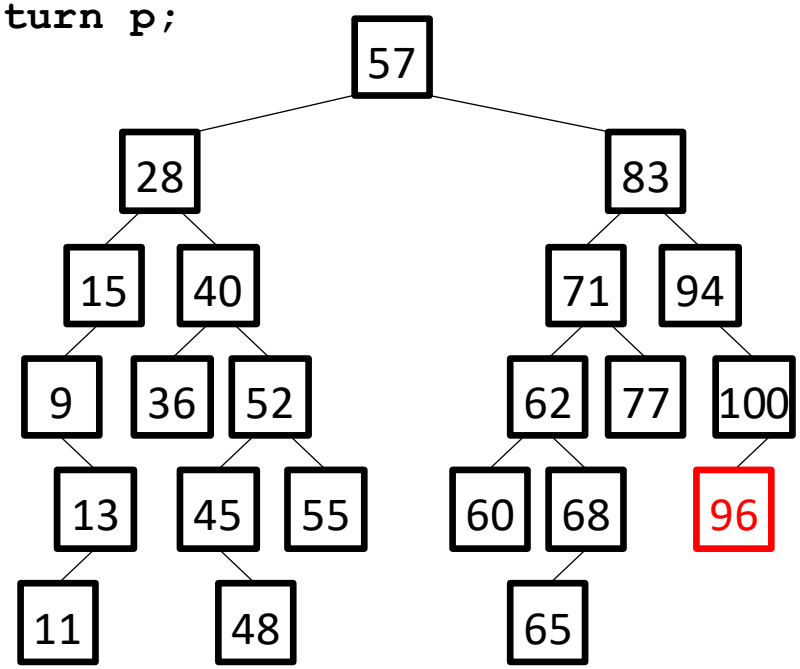
insert(65)

```
Node insert(ElementType x, Node p) {
  if (p == null) return new Node(x);
  if (x < p.data) p.left = insert(x, p.left);
  else if (x > p.data) p.right = insert(x, p.right);
  else throw exception("Duplicate key");
  return p;
}
```



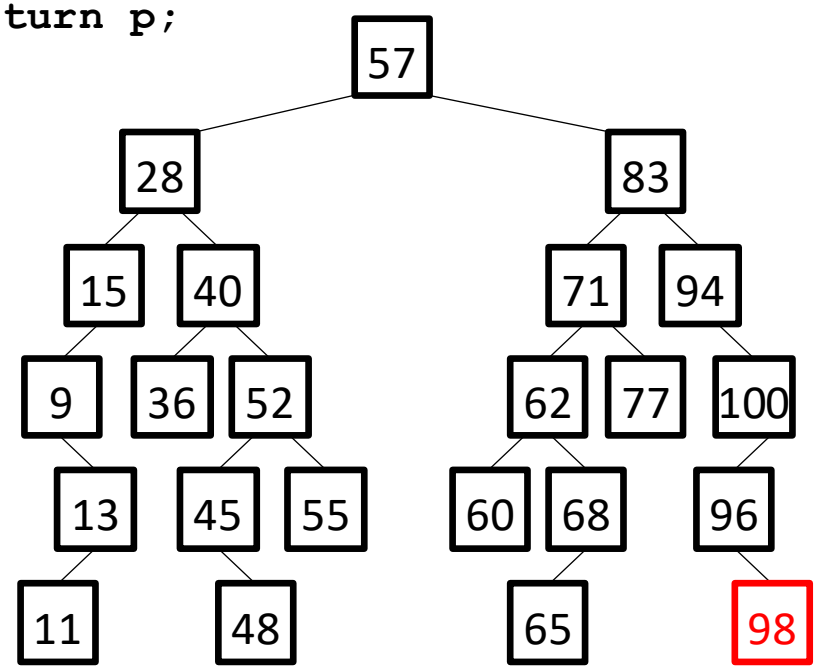
insert(96)

```
Node insert(ElementType x, Node p) {
  if (p == null) return new Node(x);
  if (x < p.data) p.left = insert(x, p.left);
  else if (x > p.data) p.right = insert(x, p.right);
  else throw exception("Duplicate key");
  return p;
}
```



insert(98)

```
Node insert(ElementType x, Node p) {
  if (p == null) return new Node(x);
  if (x < p.data) p.left = insert(x, p.left);
  else if (x > p.data) p.right = insert(x, p.right);
  else throw exception("Duplicate key");
  return p;
}
```



BST Code – remove() [successor]

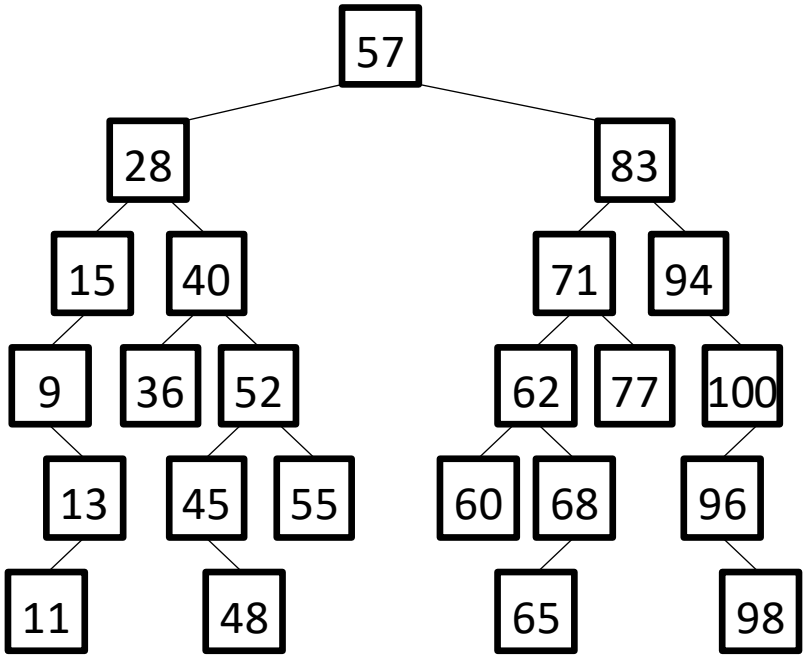
```
  void remove(ElementType x) {
    root = remove(x, root);
  }
  Node remove(ElementType x, Node p) {
    if (p == null) // search for x, end of tree
      throw exception("Key not found");
    if (x < p.data) // search for x, search left subtree
      p.left = remove(x, p.left);
    else if (x > p.data) // search for x, search right subtree
      p.right = remove(x, p.right);
    else if (p.right == null) // found x, node has the left (or no) child
      p = p.left; // "slide" the subtree (or null) up
    else if (p.left == null) // found x, node has one child, the right
      p = p.right; // "slide" the subtree up
    else { // found x, node has two children
      p.data = findMin(p.right); // find replacement (successor of x)
      p.right = remove(p.data, p.right); // remove old replacement node
    }
    return p; // set the parent's child
  }
  ElementType findMin(Node p) {
    if (p.left == null) return p.data;
    return findMin(p.left);
  }
}
```

BST Code – remove() [predecessor]

```
  void remove(ElementType x) {
    root = remove(x, root);
  }
  Node remove(ElementType x, Node p) {
    if (p == null)
      throw exception("Key not found");
    if (x < p.data)
      p.left = remove(x, p.left);
    else if (x > p.data)
      p.right = remove(x, p.right);
    else if (p.right == null)
      p = p.left;
    else if (p.left == null)
      p = p.right;
    else { // found x, node has two children
      p.data = findMax(p.left); // find replacement (predecessor of x)
      p.left = remove(p.data, p.left); // remove old replacement node
    }
    return p;
  }
  ElementType findMax(Node p) {
    if (p.right == null) return p.data;
    return findMax(p.right);
  }
}
```

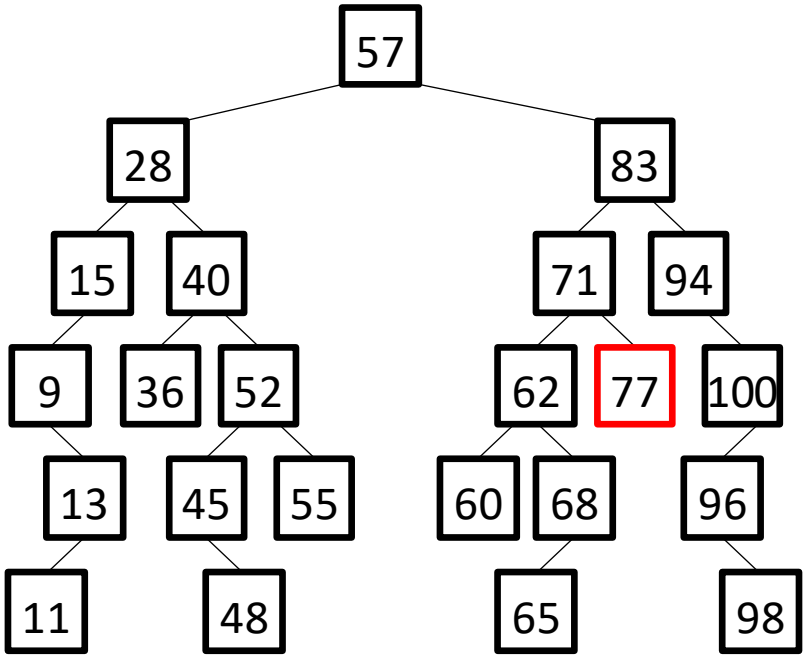
remove(77) – The Search

```
if (p == null) // search for x, end of tree
  throw exception("Key not found");
if (x < p.data) // search for x, search left subtree
  p.left = remove(x, p.left);
else if (x > p.data) // search for x, search right subtree
  p.right = remove(x, p.right);
```



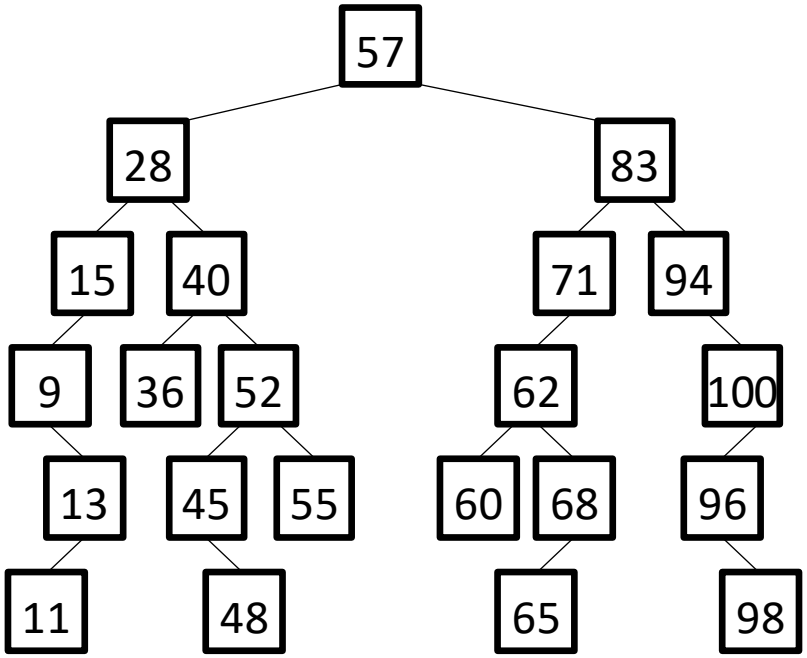
remove(77) – No Children, Chop!

```
else if (p.right == null) // found x, node has the left (or no) child
  p = p.left; // "slide" the subtree (or null) up
else if (p.left == null)
  p = p.right;
…
return p; // set the parent's child
```



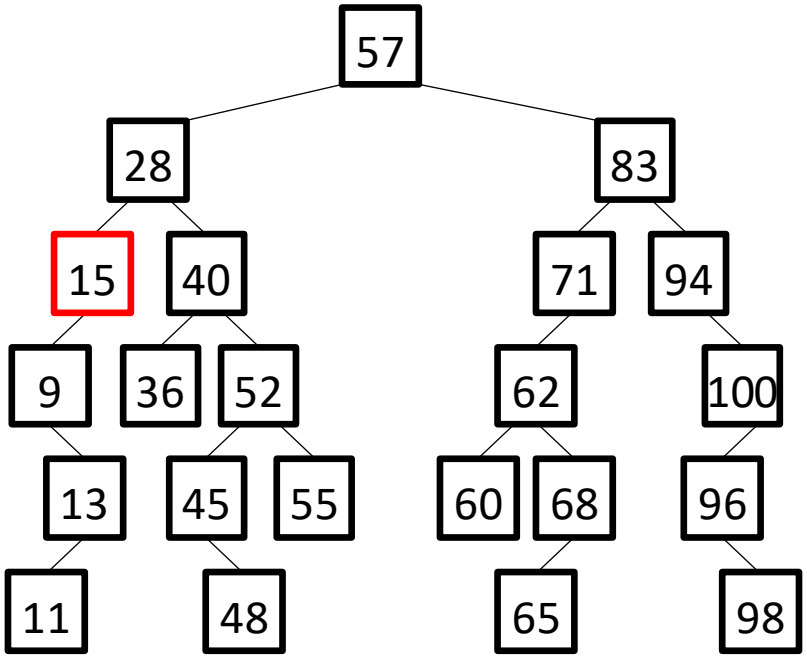
remove(15) – The Search

```
if (p == null) // search for x, end of tree
  throw exception("Key not found");
if (x < p.data) // search for x, search left subtree
  p.left = remove(x, p.left);
else if (x > p.data) // search for x, search right subtree
  p.right = remove(x, p.right);
```



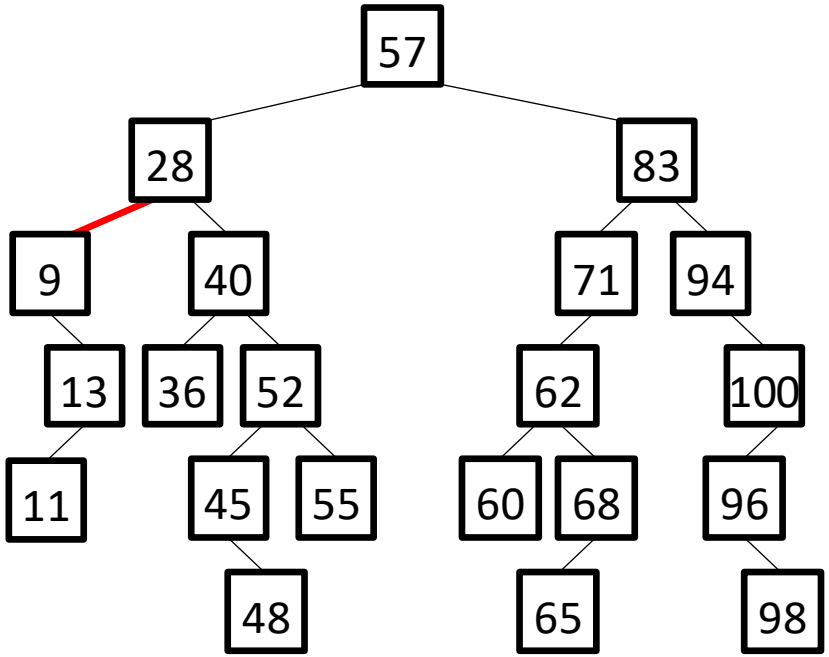
remove(15) – Left Child, Slide Up!

```
else if (p.right == null) // found x, node has the left (or no) child
  p = p.left; // "slide" the subtree (or null) up
else if (p.left == null)
  p = p.right;
…
return p; // set the parent's child
```



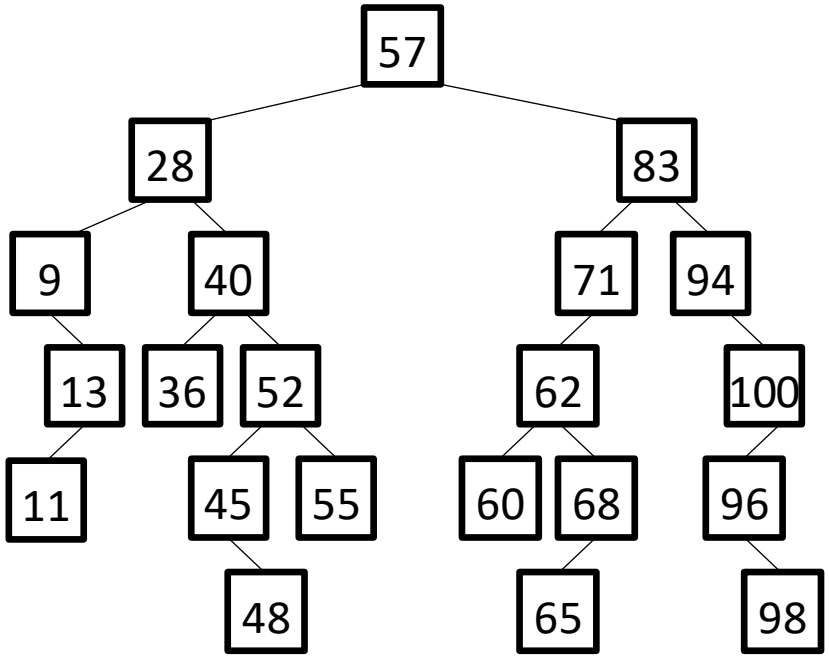
remove(15) – Left Child, Slide Up!

```
else if (p.right == null) // found x, node has the left (or no) child
  p = p.left; // "slide" the subtree (or null) up
else if (p.left == null)
  p = p.right;
…
return p; // set the parent's child
```



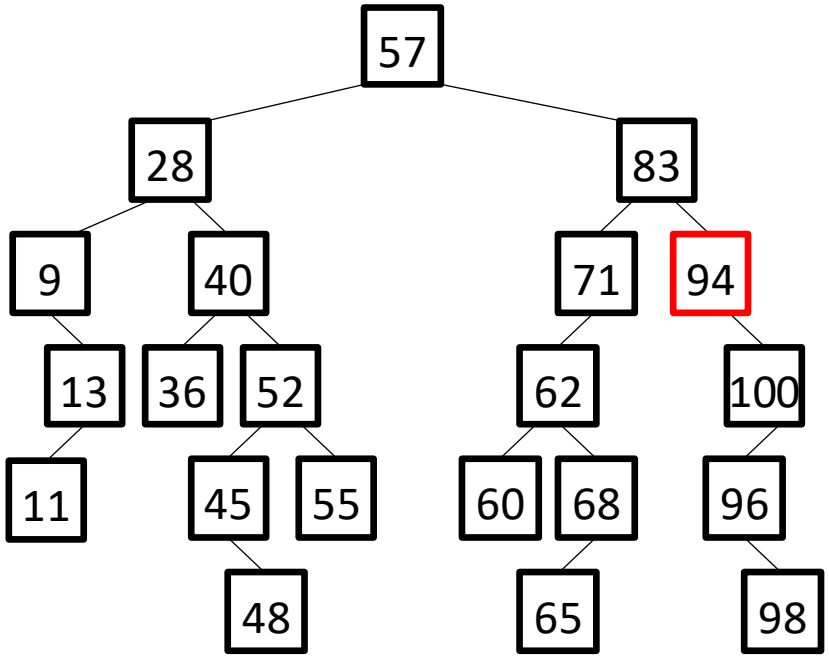
remove(94) – The Search

```
if (p == null) // search for x, end of tree
  throw exception("Key not found");
if (x < p.data) // search for x, search left subtree
  p.left = remove(x, p.left);
else if (x > p.data) // search for x, search right subtree
  p.right = remove(x, p.right);
```



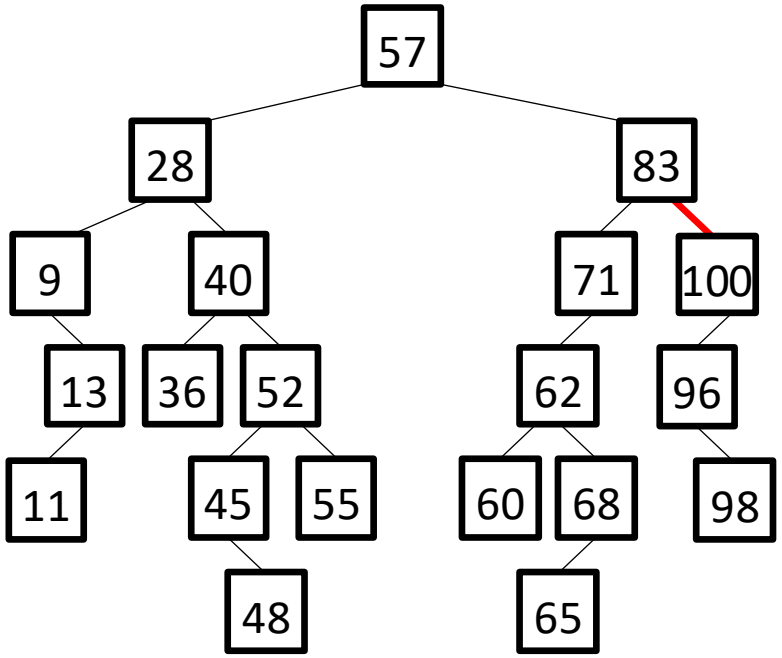
remove(94) – Right Child, Slide Up!

```
else if (p.right == null)
  p = p.left;
else if (p.left == null) // found x, node has one child, the right
  p = p.right; // "slide" the subtree up
…
return p; // set the parent's child
```



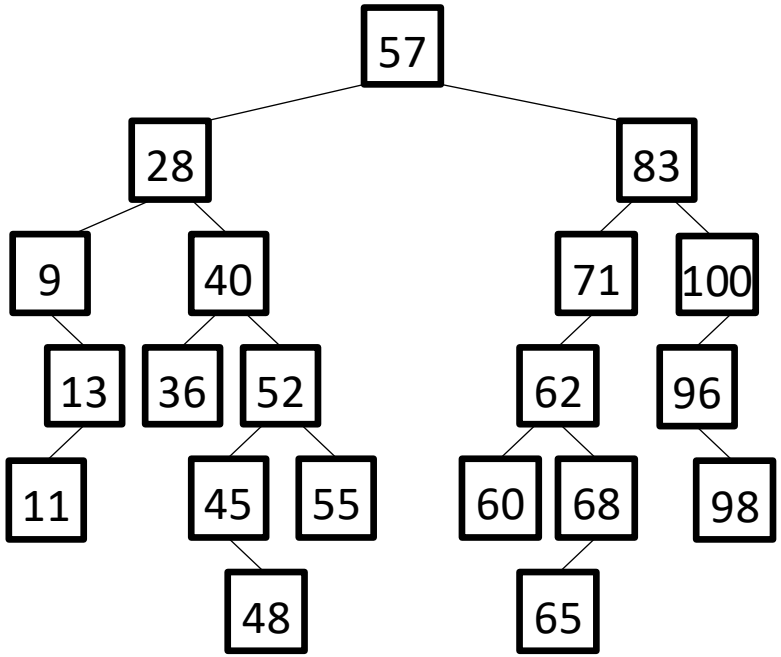
remove(94) – Right Child, Slide Up!

```
else if (p.right == null)
  p = p.left;
else if (p.left == null) // found x, node has one child, the right
  p = p.right; // "slide" the subtree up
…
return p; // set the parent's child
```



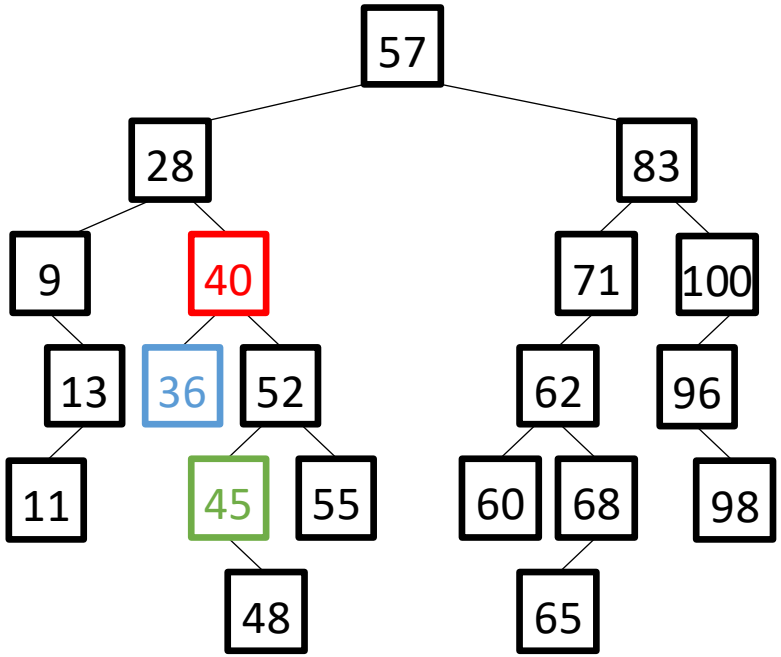
remove(40) – The Search

```
if (p == null) // search for x, end of tree
  throw exception("Key not found");
if (x < p.data) // search for x, search left subtree
  p.left = remove(x, p.left);
else if (x > p.data) // search for x, search right subtree
  p.right = remove(x, p.right);
```



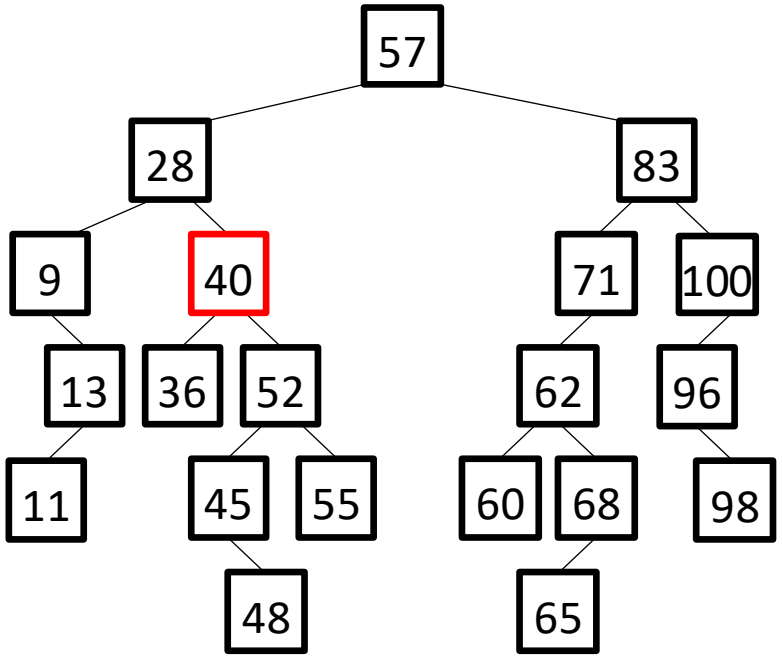
remove(40) – Note

- Inorder: 9 11 13 28 **36** **40** **45** 48 52 55 57 60 62 65 68 71 83 96 98 100
- **Successor**: Smallest (leftmost) element of the right subtree
- **Predecessor**: Largest (rightmost) element of the left subtree



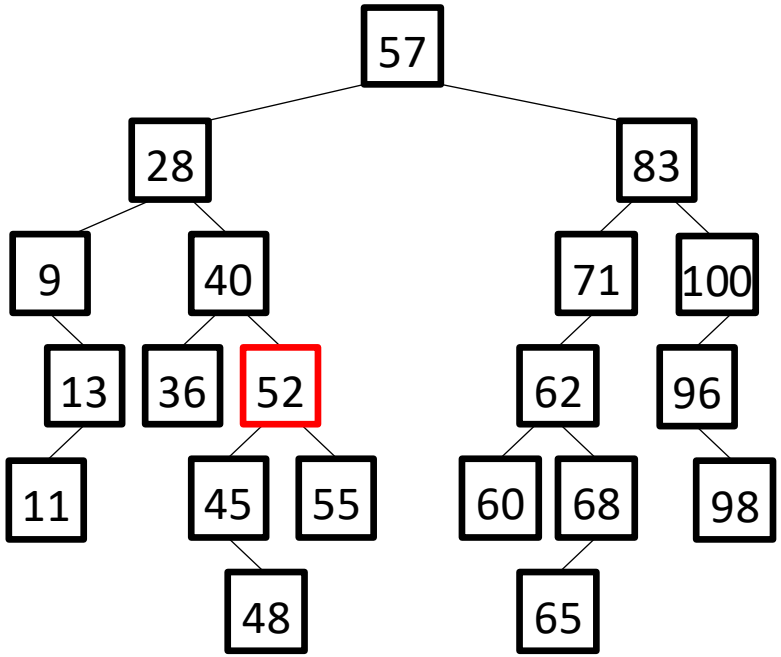
remove(40) – Children [Successor]

```
else { // found x, node has two children
  p.data = findMin(p.right); // find replacement (successor of x)
  p.right = remove(p.data, p.right); // remove old replacement node
}
…
return p; // set the parent's child
```



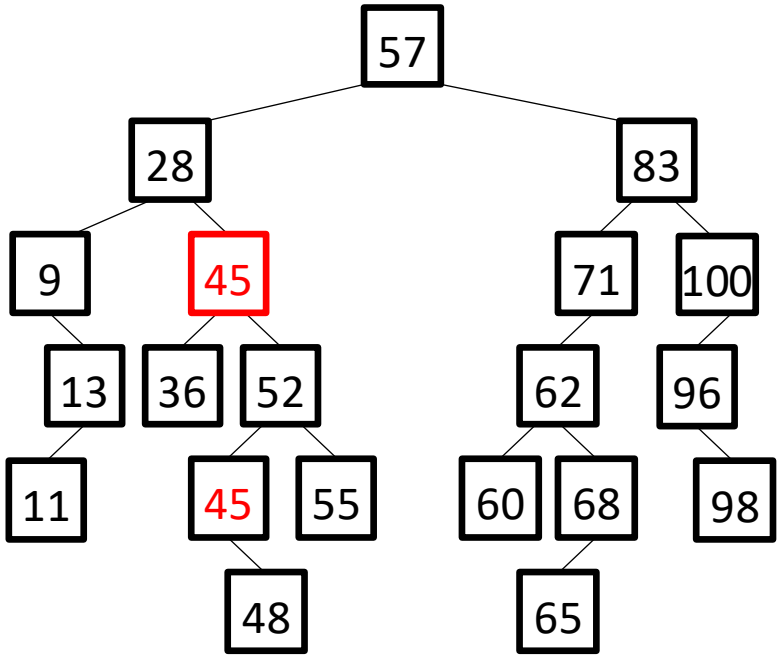
remove(40) – Children [Successor]

```
ElementType findMin(Node p) {
  if (p.left == null) return p.data;
  return findMin(p.left);
}
```



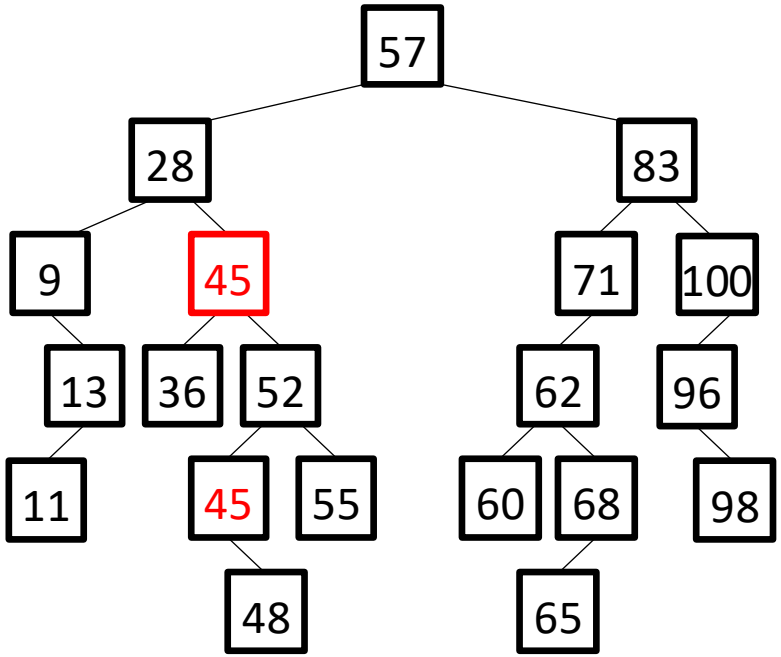
remove(40) – Children [Successor]

```
else { // found x, node has two children
  p.data = findMin(p.right); // find replacement (successor of x)
  p.right = remove(p.data, p.right); // remove old replacement node
}
…
return p; // set the parent's child
```



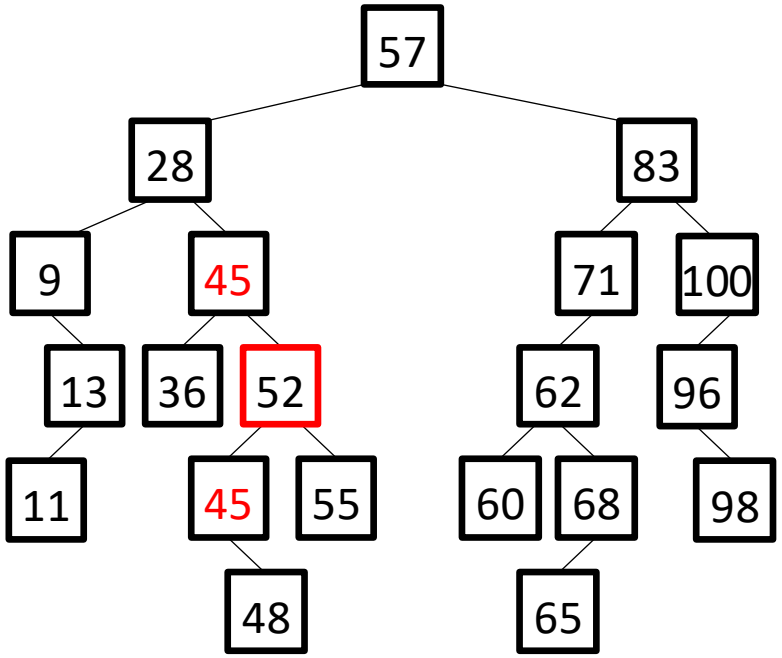
remove(40) – Children [Successor]

```
else { // found x, node has two children
  p.data = findMin(p.right); // find replacement (successor of x)
  p.right = remove(p.data, p.right); // remove old replacement node
}
…
return p; // set the parent's child
```



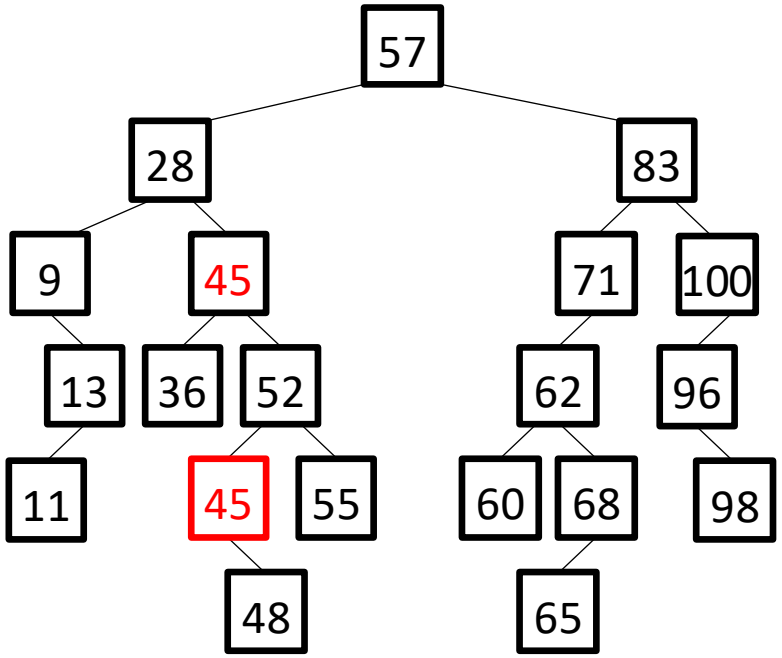
remove(45) – The Search

```
if (p == null) // search for x, end of tree
  throw exception("Key not found");
if (x < p.data) // search for x, search left subtree
  p.left = remove(x, p.left);
else if (x > p.data) // search for x, search right subtree
  p.right = remove(x, p.right);
```



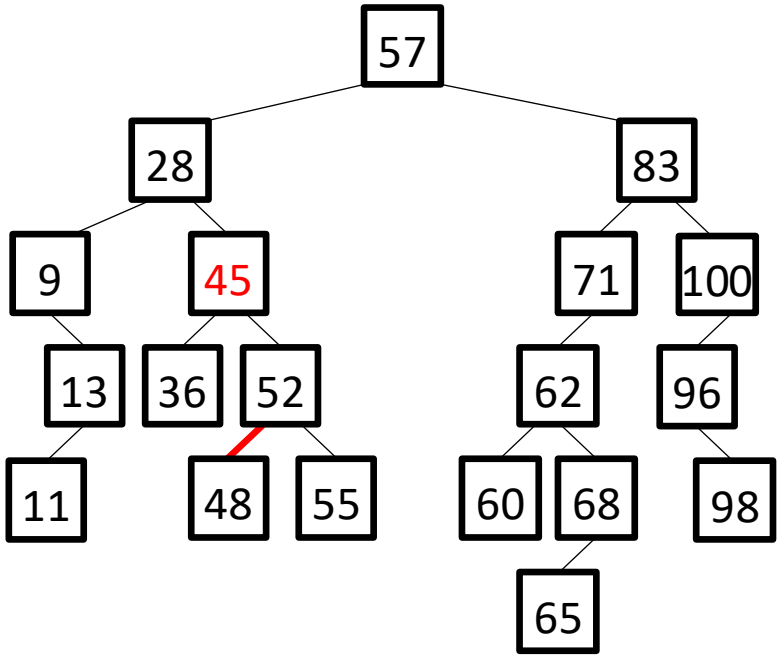
remove(45) – Right Child, Slide Up!

```
else if (p.right == null)
  p = p.left;
else if (p.left == null) // found x, node has one child, the right
  p = p.right; // "slide" the subtree up
…
return p; // set the parent's child
```



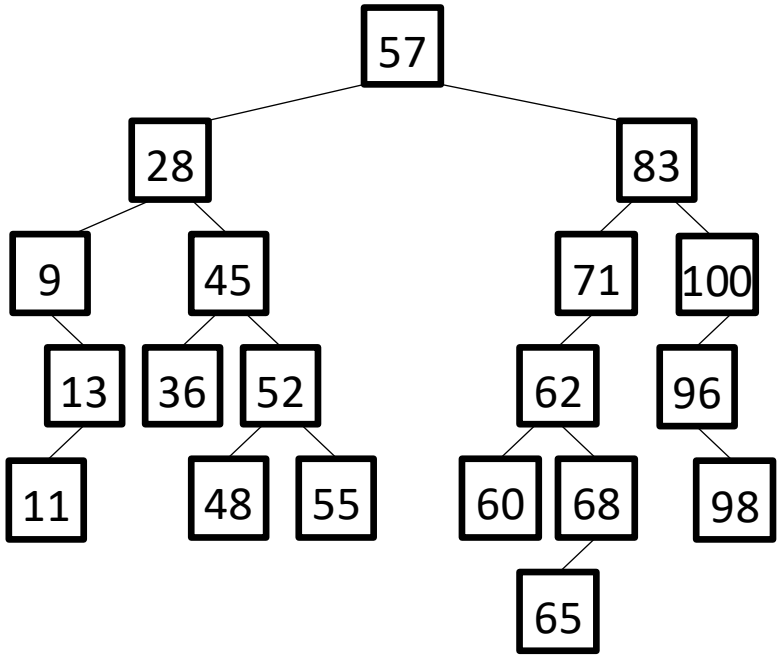
remove(45) – Right Child, Slide Up!

```
else if (p.right == null)
  p = p.left;
else if (p.left == null) // found x, node has one child, the right
  p = p.right; // "slide" the subtree up
…
return p; // set the parent's child
```



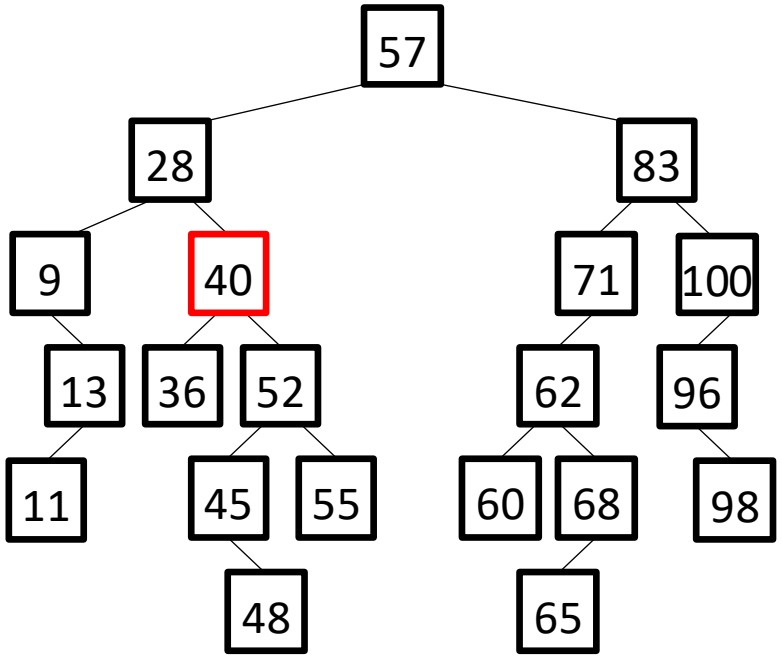
remove(40) – Finished [Successor]

```
else { // found x, node has two children
  p.data = findMin(p.right); // find replacement (successor of x)
  p.right = remove(p.data, p.right); // remove old replacement node
}
…
return p; // set the parent's child
```



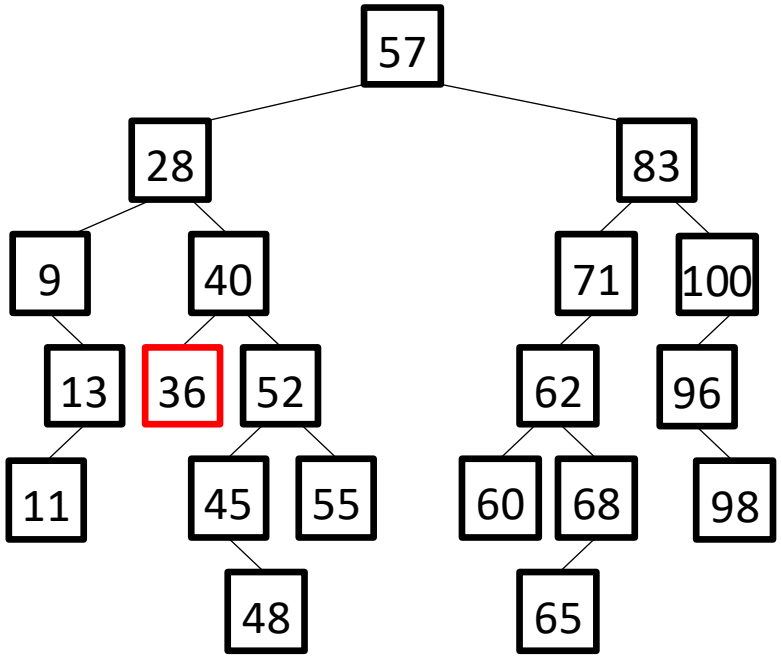
remove(40) – Children [Predecessor]

```
else { // found x, node has two children
  p.data = findMax(p.left); // find replacement (predecessor of x)
  p.left = remove(p.data, p.left); // remove old replacement node
}
…
return p; // set the parent's child
```



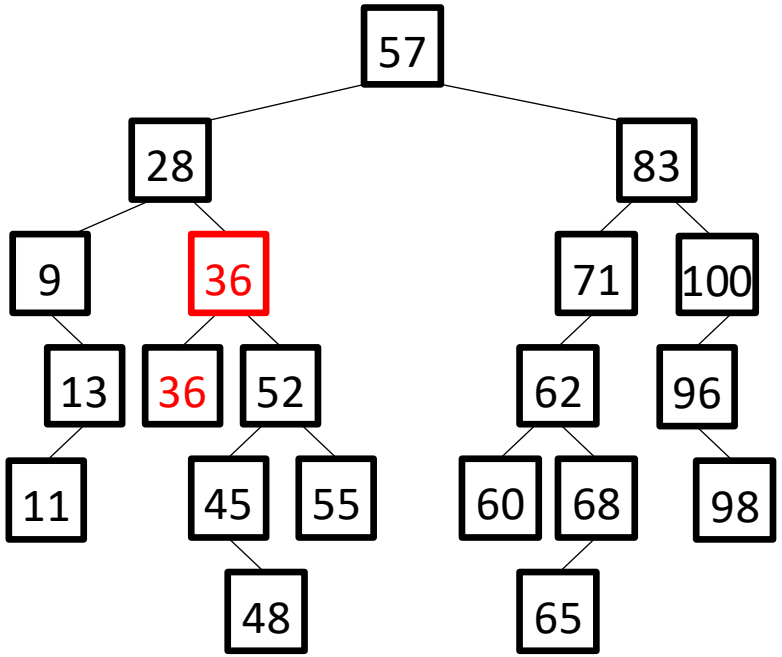
remove(40) – Children [Predecessor]

```
ElementType findMax(Node p) {
  if (p.right == null) return p.data;
  return findMax(p.right);
}
```



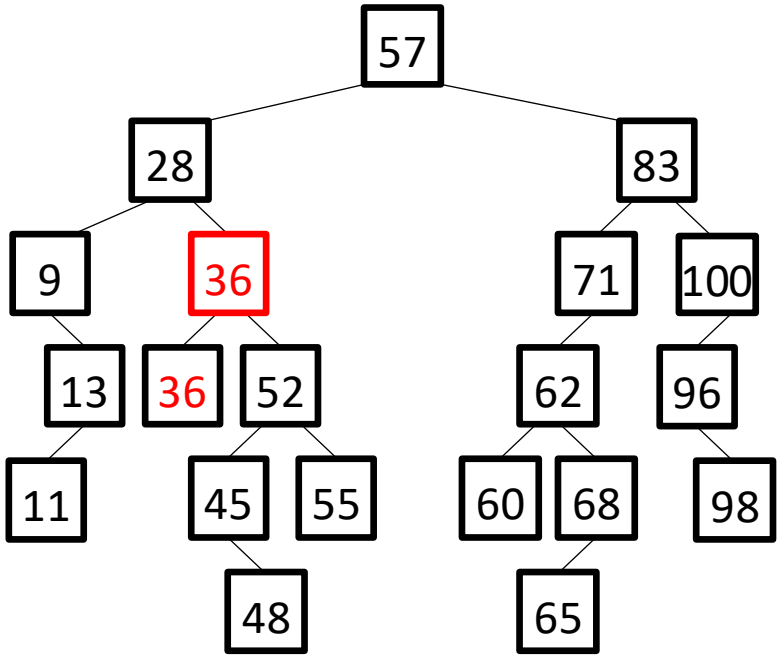
remove(40) – Children [Predecessor]

```
else { // found x, node has two children
  p.data = findMax(p.left); // find replacement (predecessor of x)
  p.left = remove(p.data, p.left); // remove old replacement node
}
…
return p; // set the parent's child
```



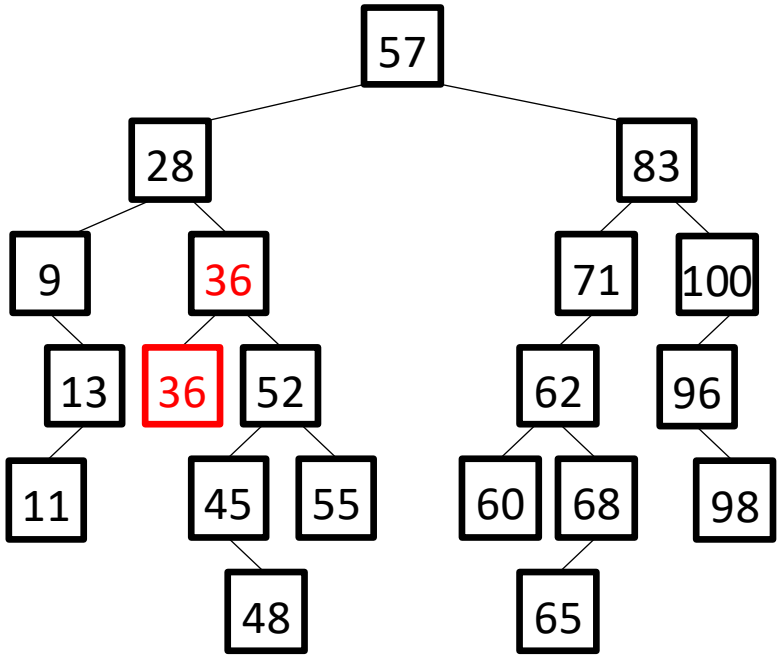
remove(40) – Children [Predecessor]

```
else { // found x, node has two children
  p.data = findMax(p.left); // find replacement (predecessor of x)
  p.left = remove(p.data, p.left); // remove old replacement node
}
…
return p; // set the parent's child
```



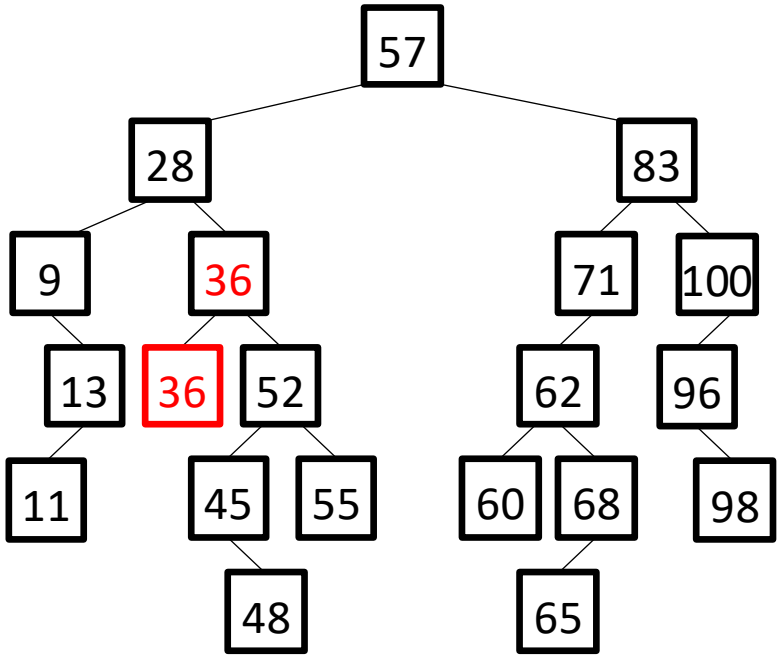
remove(36) – The Search

```
if (p == null) // search for x, end of tree
  throw exception("Key not found");
if (x < p.data) // search for x, search left subtree
  p.left = remove(x, p.left);
else if (x > p.data) // search for x, search right subtree
  p.right = remove(x, p.right);
```



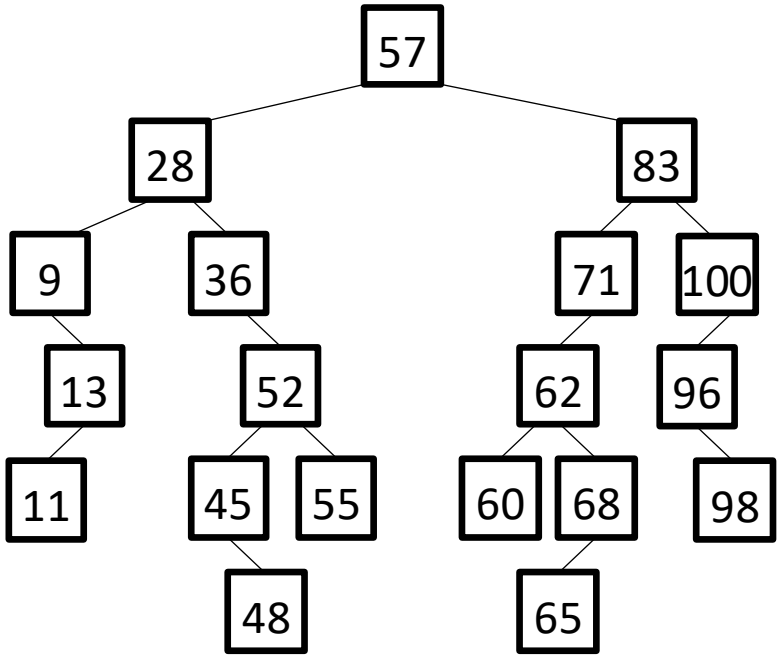
remove(36) – No Children, Chop!

```
else if (p.right == null) // found x, node has the left (or no) child
  p = p.left; // "slide" the subtree (or null) up
else if (p.left == null)
  p = p.right;
…
return p; // set the parent's child
```



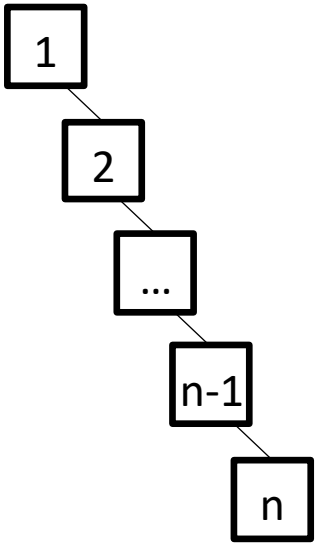
remove(40) – Finished [Predecessor]

```
else { // found x, node has two children
  p.data = findMax(p.left); // find replacement (predecessor of x)
  p.left = remove(p.data, p.left); // remove old replacement node
}
…
return p; // set the parent's child
```



Analysis

- Each BST operation (find, insert, remove) runs in time $\theta(h)$, where h = height of tree
- Worst-case time: $\theta(?)$
  - If keys are inserted in ascending order or descending order
  - Example: insert values 1 to n



Analysis

- Each BST operation (find, insert, remove) runs in time $\theta(h)$, where h = height of tree
- Worst-case time: $\theta(h) = \theta(n)$
  - If keys are inserted in ascending order or descending order
  - Example: insert values 1 to n
- Expected or average-case time: $\theta(h) = \theta(\lg n)$
  - If keys are inserted and removed in random order
- Best-case time: $\theta(1)$
  - When there is zero or one items in the tree

Analysis (cont.)

- Note: There exist other kinds of search trees such that the height and the running times are worst-case $\theta(\lg n)$, but these are topics for a later course.

Questions?